Modelos de Ordenação:
bubble sort, selection sort, insertion sort, merge sort e quick sort.

Introdução à Ordenação de Dados

Em diversos momentos do nosso dia a dia, precisamos organizar informações. Seja colocando números em ordem crescente, nomes em ordem alfabética ou classificando itens por algum critério, estamos realizando um processo de ordenação.
Na computação, essa necessidade é ainda mais frequente. Sistemas lidam constantemente com grandes volumes de dados e precisam apresentar essas informações de forma organizada para facilitar o uso, a análise e a tomada de decisão.
Para que isso seja possível, não basta apenas armazenar os dados. É necessário definir como eles serão organizados. Esse processo é realizado por meio de algoritmos de ordenação, que são conjuntos de regras que determinam como os dados devem ser rearranjados.
Existem diferentes formas de ordenar dados, cada uma com sua própria lógica e eficiência. Algumas estratégias são mais simples e fáceis de entender, enquanto outras são mais rápidas e utilizadas em aplicações reais.
Compreender esses modelos é fundamental, pois a escolha do algoritmo de ordenação impacta diretamente no desempenho de um sistema, principalmente quando trabalhamos com grandes quantidades de dados.

Onde a Ordenação é Aplicada

Após compreender o conceito de ordenação, é importante perceber que esse não é apenas um conteúdo teórico. A ordenação está presente em praticamente todos os sistemas computacionais.
Sempre que um sistema precisa organizar, priorizar ou facilitar o acesso a informações, algum algoritmo de ordenação está sendo utilizado.
Podemos observar aplicações diretas em diversos contextos:

- Sistemas acadêmicos que organizam alunos por nome, matrícula ou nota
- Lojas virtuais que exibem produtos por preço, relevância ou avaliação
- Bancos de dados que estruturam registros para otimizar consultas
- Mecanismos de busca que apresentam resultados ordenados por relevância

- Sistemas corporativos que processam grandes volumes de informações

Além disso, a ordenação muitas vezes é etapa obrigatória para outros algoritmos funcionarem corretamente. Um exemplo clássico é a busca binária, que só funciona com dados previamente ordenados.

Por que existem vários algoritmos?

Embora o problema seja o mesmo, organizar dados, existem diferentes formas de resolver essa tarefa.
Cada algoritmo foi desenvolvido pensando em cenários específicos:

- Alguns são mais simples e ideais para aprendizado e pequenos volumes
- Outros são mais eficientes e utilizados em sistemas reais
- Alguns funcionam melhor com dados já parcialmente organizados
- Outros são mais adequados para grandes volumes de dados

Isso significa que não existe um único algoritmo ideal para todas as situações.

Impacto na prática

A escolha do algoritmo influencia diretamente:

- O tempo que o sistema leva para responder
- O consumo de recursos computacionais
- A escalabilidade da aplicação

Em pequenos conjuntos de dados, a diferença pode ser imperceptível. Porém, em sistemas reais, com milhares ou milhões de registros, escolher o algoritmo errado pode gerar perda significativa de desempenho.

Modelos de Ordenação que serão estudados

Para resolver o problema de ordenação, existem diferentes algoritmos, cada um com uma estratégia própria de organização dos dados. Neste conteúdo, serão trabalhados cinco modelos clássicos:

Algoritmos mais simples (base para compreensão):

- Bubble Sort
- Selection Sort
- Insertion Sort

Esses algoritmos são importantes para entender a lógica de ordenação e como os dados podem ser manipulados passo a passo.

Algoritmos mais eficientes (utilizados em sistemas reais):

- Merge Sort
- Quick Sort

Esses modelos utilizam estratégias mais avançadas e apresentam melhor desempenho, principalmente quando aplicados a grandes volumes de dados.

Bubble Sort

O Bubble Sort, também conhecido como método da bolha, é um algoritmo de ordenação baseado na comparação de elementos vizinhos em um vetor.
Seu funcionamento é simples: o algoritmo percorre o vetor comparando dois elementos lado a lado e realiza a troca de posição sempre que eles estiverem fora de ordem. Esse processo se repete várias vezes até que todos os elementos estejam organizados.
A cada passagem pelo vetor, o maior elemento tende a "subir" para o final, comportamento que dá origem ao nome do método, fazendo uma analogia com uma bolha que sobe em um líquido.
O Bubble Sort não possui um autor específico definido, sendo um dos primeiros algoritmos ensinados na área da computação. Sua principal utilização é didática, pois permite compreender de forma clara conceitos fundamentais como comparações, trocas de valores e uso de estruturas de repetição.
Entre suas principais características, destaca-se a facilidade de implementação e o fato de trabalhar com comparações locais, sempre entre elementos vizinhos. Por outro lado, apresenta um alto número de comparações e trocas, o que o torna pouco eficiente quando aplicado a grandes volumes de dados.
Uma melhoria comum nesse algoritmo é a utilização de uma variável de controle, geralmente chamada de troca. Essa variável permite verificar se houve alguma troca durante uma passagem pelo vetor. Caso nenhuma troca ocorra, significa que o vetor já está ordenado, permitindo que o algoritmo seja encerrado antes de executar todas as iterações previstas, tornando-o mais eficiente em casos favoráveis.

Exemplo detalhado (passo a passo)

Para compreender melhor o funcionamento do Bubble Sort, é importante visualizar como as comparações e trocas acontecem durante a execução do algoritmo.

Neste exemplo, será utilizado um vetor com 8 elementos desordenados. A ideia é acompanhar, passo a passo, cada comparação realizada e identificar quando ocorre uma troca de valores.
Durante a execução, observe que:

- Os elementos são comparados sempre dois a dois (elementos vizinhos)
- Quando necessário, ocorre a troca de posição
- A cada passagem, o maior valor vai sendo deslocado para o final do vetor
- O processo se repete até que nenhuma troca seja necessária

Essa análise detalhada permite entender claramente a lógica do algoritmo e como ele evolui até que o vetor esteja completamente ordenado.

Considere o vetor: [5, 3, 4, 1, 2, 8, 7, 6]

Passagem 1

(5,3) → troca → [3,5,4,1,2,8,7,6]
(5,4) → troca → [3,4,5,1,2,8,7,6]
(5,1) → troca → [3,4,1,5,2,8,7,6]
(5,2) → troca → [3,4,1,2,5,8,7,6]
(5,8) → ok
(8,7) → troca → [3,4,1,2,5,7,8,6]
(8,6) → troca → [3,4,1,2,5,7,6,8]

Maior valor (8) já está na última posição

Passagem 2

(3,4) → ok
(4,1) → troca → [3,1,4,2,5,7,6,8]
(4,2) → troca → [3,1,2,4,5,7,6,8]
(4,5) → ok
(5,7) → ok
(7,6) → troca → [3,1,2,4,5,6,7,8]

Segundo maior (7) já está correto

Passagem 3

(3,1) → troca → [1,3,2,4,5,6,7,8]
(3,2) → troca → [1,2,3,4,5,6,7,8]
restante → ok

Vetor praticamente ordenado

Passagem 4

Nenhuma troca ocorre → algoritmo encerra

Implementação em C ANSI

```
#include <stdio.h>

#define TAM 8

int main(void){

    int v[TAM];
    int i, n, aux, troca;

    // Entrada de dados
    for(i = 0; i < TAM; i++){
        printf("Digite o %i numero: ", i+1);
        scanf("%i", &v[i]);
    }

    // Bubble Sort
    n = 1;
    troca = 1;

    while(n <= TAM && troca == 1){
        troca = 0;

        for(i = 0; i < TAM - 1; i++){
            if(v[i] > v[i + 1]){
                troca = 1;

                aux = v[i];
                v[i] = v[i + 1];
                v[i + 1] = aux;
            }
        }

        n++;
    }

    // Saída de dados
    printf("\nVetor ordenado:\n");
    for(i = 0; i < TAM; i++){
        printf("%i ", v[i]);
    }
```

```
    return 0;
}
```

Selection Sort

O Selection Sort é um algoritmo de ordenação baseado em uma ideia simples: encontrar o menor valor e colocá-lo na posição correta do vetor.
Imagine que você precisa organizar uma lista de números. Em vez de ficar comparando e trocando vários elementos o tempo todo, você primeiro procura o menor número de todos e o coloca na primeira posição. Depois disso, procura o segundo menor valor e o coloca na segunda posição, e assim sucessivamente até que toda a lista esteja ordenada.
O algoritmo funciona exatamente dessa forma. Ele divide o vetor em duas partes: uma parte já organizada e outra ainda desorganizada. A cada passo, percorre a parte não ordenada, identifica o menor elemento e realiza uma única troca para posicioná-lo corretamente.
Diferente do Bubble Sort, que faz várias trocas ao longo do processo, o Selection Sort é mais controlado, realizando apenas uma troca por etapa. Isso reduz a quantidade de movimentações, embora o número de comparações continue sendo alto.
Por ser simples e direto, é muito utilizado no ensino para ajudar a entender como funciona o processo de ordenação. No entanto, não é indicado para grandes volumes de dados, pois seu desempenho não é eficiente em comparação com algoritmos mais avançados.
Sua lógica consiste em, para cada posição do vetor, assumir que aquele elemento é o menor, percorrer o restante do vetor em busca de um valor menor e, caso encontre, atualizar essa posição e realizar a troca.

Exemplo detalhado (passo a passo)

Para compreender melhor o funcionamento do Selection Sort, é importante visualizar como as comparações e trocas acontecem durante a execução do algoritmo.
Neste exemplo, será utilizado um vetor com 8 elementos desordenados. A ideia é acompanhar, passo a passo, cada comparação realizada e identificar como o menor valor é selecionado e posicionado corretamente.

Durante a execução, observe que:

- O algoritmo percorre todo o restante do vetor em busca do menor elemento
- O menor valor encontrado é selecionado a cada passagem
- A troca ocorre apenas uma vez por etapa, posicionando corretamente o elemento
- A cada passagem, a parte inicial do vetor vai ficando ordenada

- O processo se repete até que todos os elementos estejam organizados

Essa análise detalhada permite entender claramente a lógica do algoritmo e como ele evolui até que o vetor esteja completamente ordenado.

Considere o vetor: [5, 3, 4, 1, 2, 8, 7, 6]

Passagem 1

Comparações (buscando o menor):
(5,3) → novo menor = 3
(3,4) → ok
(3,1) → novo menor = 1
(1,2) → ok
(1,8) → ok
(1,7) → ok
(1,6) → ok

Troca:
(5 ↔ 1) → [1,3,4,5,2,8,7,6]
Menor valor (1) já está na primeira posição

Passagem 2

Comparações:
(3,4) → ok
(3,5) → ok
(3,2) → novo menor = 2
(2,8) → ok
(2,7) → ok
(2,6) → ok

Troca:
(3 ↔ 2) → [1,2,4,5,3,8,7,6]
Segundo menor (2) já está correto

Passagem 3

Comparações:
(4,5) → ok
(4,3) → novo menor = 3
(3,8) → ok
(3,7) → ok
(3,6) → ok

Troca:
(4 ↔ 3) → [1,2,3,5,4,8,7,6]
Terceiro menor (3) já está correto

Passagem 4

Comparações:
(5,4) → novo menor = 4
(4,8) → ok
(4,7) → ok
(4,6) → ok

Troca:
(5 ↔ 4) → [1,2,3,4,5,8,7,6]
Parte inicial já ordenada

Passagem 5

Comparações:
(5,8) → ok
(5,7) → ok
(5,6) → ok
Troca:
sem troca → [1,2,3,4,5,8,7,6]
Nenhuma alteração necessária

Passagem 6

Comparações:
(8,7) → novo menor = 7
(7,6) → novo menor = 6

Troca:
(8 ↔ 6) → [1,2,3,4,5,6,7,8]
Elementos finais ajustados

Passagem 7

Comparações:
(7,8) → ok
Troca:
sem troca → [1,2,3,4,5,6,7,8]

Resultado final: [1, 2, 3, 4, 5, 6, 7, 8]

Implementação em C ANSI

```
#include <stdio.h>
```

```c
#define TAM 8

int main(void){

    int v[TAM];
    int i, j, min, aux;

    // Entrada de dados
    for(i = 0; i < TAM; i++){
        printf("Digite o %i numero: ", i+1);
        scanf("%i", &v[i]);
    }

    // Selection Sort
    for(i = 0; i < TAM - 1; i++){

        min = i;

        for(j = i + 1; j < TAM; j++){
            if(v[j] < v[min]){
                min = j;
            }
        }

        if(min != i){
            aux = v[i];
            v[i] = v[min];
            v[min] = aux;
        }
    }

    // Saída de dados
    printf("\nVetor ordenado:\n");
    for(i = 0; i < TAM; i++){
        printf("%i ", v[i]);
    }

    return 0;
}
```

Insertion Sort

O Insertion Sort é um algoritmo de ordenação baseado na ideia de inserir cada elemento na posição correta dentro de uma parte já ordenada do vetor.
Uma forma simples de entender esse algoritmo é imaginar como organizamos cartas na mão em um jogo. Pegamos uma carta por vez e a inserimos na posição correta em relação às cartas que já estão

organizadas. O funcionamento do algoritmo segue exatamente essa lógica.
Ele considera que o primeiro elemento já está ordenado e, a partir do segundo elemento, compara com os anteriores, deslocando os valores maiores para a direita para abrir espaço e inserir o elemento na posição correta.
Diferente do Bubble Sort e do Selection Sort, o Insertion Sort não realiza trocas diretas entre elementos. Em vez disso, ele trabalha deslocando os valores até encontrar o local adequado para inserção, o que torna seu comportamento mais organizado.
Esse algoritmo também não possui um autor específico definido e é amplamente utilizado no ensino por ser intuitivo e muito próximo de situações reais do dia a dia.
Entre suas principais características, destaca-se a facilidade de entendimento e implementação, além de apresentar um bom desempenho quando aplicado a listas pequenas ou que já estão parcialmente ordenadas. Por outro lado, não é indicado para grandes volumes de dados, pois seu desempenho se torna limitado.
Sua lógica consiste em percorrer o vetor a partir do segundo elemento e, para cada posição, selecionar o valor atual, comparar com os elementos anteriores, deslocar os maiores para a direita e inserir o elemento na posição correta

Exemplo detalhado (passo a passo)

Para compreender melhor o funcionamento do Insertion Sort, é importante visualizar como as comparações e deslocamentos acontecem durante a execução do algoritmo.

Neste exemplo, será utilizado um vetor com 8 elementos desordenados. A ideia é acompanhar, passo a passo, cada comparação realizada e identificar como cada elemento é inserido na posição correta dentro da parte já ordenada do vetor.

Durante a execução, observe que:

- O algoritmo considera o primeiro elemento como já ordenado
- A cada passo, um novo elemento é selecionado para inserção
- Os elementos maiores são deslocados para a direita
- O valor selecionado é inserido na posição correta
- A cada passagem, a parte inicial do vetor vai ficando ordenada
- O processo se repete até que todos os elementos estejam organizados

Essa análise detalhada permite entender claramente a lógica do algoritmo e como ele evolui até que o vetor esteja completamente ordenado.

Considere o vetor: [5, 3, 4, 1, 2, 8, 7, 6]

Passagem 1 (inserindo o 3)

Comparações e deslocamentos:
(5,3) → desloca → [5,5,4,1,2,8,7,6]
insere 3 → [3,5,4,1,2,8,7,6]
Subvetor ordenado: [3,5]

Passagem 2 (inserindo o 4)

Comparações e deslocamentos:
(5,4) → desloca → [3,5,5,1,2,8,7,6]
(3,4) → ok
insere 4 → [3,4,5,1,2,8,7,6]
Subvetor ordenado: [3,4,5]

Passagem 3 (inserindo o 1)

Comparações e deslocamentos:
(5,1) → desloca → [3,4,5,5,2,8,7,6]
(4,1) → desloca → [3,4,4,5,2,8,7,6]
(3,1) → desloca → [3,3,4,5,2,8,7,6]
insere 1 → [1,3,4,5,2,8,7,6]
Subvetor ordenado: [1,3,4,5]

Passagem 4 (inserindo o 2)

Comparações e deslocamentos:
(5,2) → desloca → [1,3,4,5,5,8,7,6]
(4,2) → desloca → [1,3,4,4,5,8,7,6]
(3,2) → desloca → [1,3,3,4,5,8,7,6]
(1,2) → ok
insere 2 → [1,2,3,4,5,8,7,6]
Subvetor ordenado: [1,2,3,4,5]

Passagem 5 (inserindo o 8)

Comparações:
(5,8) → ok
insere 8 → [1,2,3,4,5,8,7,6]
Subvetor ordenado: [1,2,3,4,5,8]

Passagem 6 (inserindo o 7)

Comparações e deslocamentos:
(8,7) → desloca → [1,2,3,4,5,8,8,6]

(5,7) → ok
insere 7 → [1,2,3,4,5,7,8,6]
Subvetor ordenado: [1,2,3,4,5,7,8]

Passagem 7 (inserindo o 6)

Comparações e deslocamentos:
(8,6) → desloca → [1,2,3,4,5,7,8,8]
(7,6) → desloca → [1,2,3,4,5,7,7,8]
(5,6) → ok
insere 6 → [1,2,3,4,5,6,7,8]

Resultado final: [1, 2, 3, 4, 5, 6, 7, 8]

Implementação em C ANSI

```
#include <stdio.h>

#define TAM 8

int main(void){

    int v[TAM];
    int i, j, chave;

    // Entrada de dados
    for(i = 0; i < TAM; i++){
        printf("Digite o %i numero: ", i+1);
        scanf("%i", &v[i]);
    }

    // Insertion Sort
    for(i = 1; i < TAM; i++){

        chave = v[i];
        j = i - 1;

        while(j >= 0 && v[j] > chave){
            v[j + 1] = v[j];
            j--;
        }

        v[j + 1] = chave;
    }

    // Saída de dados
    printf("\nVetor ordenado:\n");
    for(i = 0; i < TAM; i++){
        printf("%i ", v[i]);
```

```
    }

    return 0;
}
```

Merge Sort

O Merge Sort é um algoritmo de ordenação baseado na estratégia de dividir para conquistar. Diferente dos algoritmos anteriores, que trabalham comparando e reorganizando os elementos diretamente no vetor, o Merge Sort utiliza uma abordagem mais estruturada, dividindo o problema em partes menores para facilitar a ordenação.
Para entender melhor, imagine que você tem uma lista grande de números desordenados. Em vez de tentar organizar tudo de uma vez, o algoritmo divide essa lista em duas partes, depois divide novamente essas partes, repetindo esse processo até que cada subparte contenha apenas um único elemento. A partir desse ponto, o algoritmo começa a juntar essas partes, organizando os elementos durante a junção.
Esse processo de divisão e combinação permite que o Merge Sort organize os dados de forma mais eficiente, especialmente quando trabalhamos com grandes volumes de informação. Enquanto algoritmos como Bubble Sort, Selection Sort e Insertion Sort realizam várias comparações locais e trocas diretas, o Merge Sort organiza os dados de maneira mais global e estruturada.
Por utilizar essa estratégia, o Merge Sort apresenta um desempenho superior e previsível, sendo amplamente utilizado em sistemas reais que exigem eficiência e estabilidade na ordenação de dados.

Exemplo detalhado (passo a passo)

Para compreender melhor o funcionamento do Merge Sort, é importante visualizar como acontecem as etapas de divisão e junção durante a execução do algoritmo.
Neste exemplo, será utilizado um vetor com 8 elementos desordenados. A ideia é acompanhar, passo a passo, como o vetor é dividido em partes menores até restarem elementos isolados e, em seguida, como essas partes são reunidas de forma ordenada.
Durante a execução, observe que:

- O algoritmo divide o vetor em partes menores sucessivamente
- A divisão continua até que cada parte tenha apenas um elemento
- Após isso, inicia-se o processo de junção das partes
- Durante a junção, os elementos são comparados e organizados
- A cada etapa, subvetores menores vão sendo combinados em ordem
- O processo se repete até que todo o vetor esteja completamente ordenado

Essa análise detalhada permite entender claramente a lógica do algoritmo e como ele evolui até que o vetor esteja completamente ordenado.

Considere o vetor: [5, 3, 4, 1, 2, 8, 7, 6]

Passagem 1 (Divisão)

Dividindo o vetor:
[5, 3, 4, 1] | [2, 8, 7, 6]

Passagem 2 (Divisão)

[5, 3] | [4, 1] | [2, 8] | [7, 6]

Passagem 3 (Divisão)

[5] [3] | [4] [1] | [2] [8] | [7] [6]
Vetores com apenas 1 elemento já estão ordenados

Passagem 4 (Junção)

Comparações e ordenação:
(5,3) → [3,5]
(4,1) → [1,4]
(2,8) → [2,8]
(7,6) → [6,7]

Resultado:
[3,5] | [1,4] | [2,8] | [6,7]

Passagem 5 (Junção)

Comparações e ordenação:
(3,1) → [1,...]
(3,4) → [1,3,...]
(5,4) → [1,3,4,...]
restante → [1,3,4,5]

(2,6) → [2,...]
(8,6) → [2,6,...]
(8,7) → [2,6,7,...]
restante → [2,6,7,8]

Resultado:
[1,3,4,5] | [2,6,7,8]

Passagem 6 (Junção Final)

Comparações:
(1,2) → [1,...]
(3,2) → [1,2,...]
(3,6) → [1,2,3,...]
(4,6) → [1,2,3,4,...]
(5,6) → [1,2,3,4,5,...]
restante → [1,2,3,4,5,6,7,8]

Resultado final: [1, 2, 3, 4, 5, 6, 7, 8]

Implementação em C ANSI

```
#include <stdio.h>

#define TAM 8

// Função para juntar (merge) os subvetores
void merge(int *v, int inicio, int meio, int fim){

    int i, j, k;
    int temp[TAM];

    i = inicio;
    j = meio + 1;
    k = inicio;

    // Intercala os elementos
    while(i <= meio && j <= fim){
        if(v[i] < v[j]){
            temp[k] = v[i];
            i++;
        } else {
            temp[k] = v[j];
            j++;
        }
        k++;
    }

    // Copia os elementos restantes
    while(i <= meio){
        temp[k] = v[i];
        i++;
        k++;
    }

    while(j <= fim){
        temp[k] = v[j];
```

```
        j++;
        k++;
    }

    // Copia de volta para o vetor original
    for(i = inicio; i <= fim; i++){
        v[i] = temp[i];
    }
}

// Função recursiva do Merge Sort
void mergeSort(int *v, int inicio, int fim){

    int meio;

    if(inicio < fim){
        meio = (inicio + fim) / 2;

        mergeSort(v, inicio, meio);
        mergeSort(v, meio + 1, fim);

        merge(v, inicio, meio, fim);
    }
}

int main(void){

    int v[TAM];
    int i;

    // Entrada de dados
    for(i = 0; i < TAM; i++){
        printf("Digite o %i numero: ", i+1);
        scanf("%i", &v[i]);
    }

    // Merge Sort
    mergeSort(v, 0, TAM - 1);

    // Saída de dados
    printf("\nVetor ordenado:\n");
    for(i = 0; i < TAM; i++){
        printf("%i ", v[i]);
    }

    return 0;
}
```

Quick Sort

O Quick Sort é um algoritmo de ordenação baseado também na estratégia de dividir para conquistar, assim como o Merge Sort, porém com uma abordagem diferente na forma de organizar os dados.
A ideia principal do Quick Sort é escolher um elemento do vetor, chamado de pivô, e reorganizar os demais elementos de forma que todos os valores menores que o pivô fiquem à sua esquerda e todos os valores maiores fiquem à sua direita. Após essa reorganização, o pivô já estará na posição correta dentro do vetor.
A partir disso, o algoritmo repete o mesmo processo para as partes esquerda e direita, dividindo o problema em subproblemas menores até que todo o vetor esteja ordenado.
Diferente dos algoritmos mais simples, como Bubble Sort, Selection Sort e Insertion Sort, o Quick Sort não trabalha com comparações locais ou inserções diretas, mas sim com particionamento dos dados, o que torna seu desempenho muito mais eficiente na maioria dos casos.
Esse algoritmo foi desenvolvido por Tony Hoare em 1959 e é amplamente utilizado em sistemas reais devido à sua alta eficiência e bom desempenho médio. Em muitos cenários, ele é considerado um dos algoritmos de ordenação mais rápidos.
Por utilizar essa estratégia de divisão baseada em pivô, o Quick Sort consegue organizar grandes volumes de dados de forma eficiente, sendo uma escolha comum quando se busca desempenho em aplicações reais.

Exemplo detalhado (passo a passo)

Para compreender melhor o funcionamento do Quick Sort, é importante visualizar como acontecem as etapas de escolha do pivô e particionamento dos elementos durante a execução do algoritmo.
Neste exemplo, será utilizado um vetor com 8 elementos desordenados. A ideia é acompanhar, passo a passo, como um elemento é escolhido como pivô e como os demais valores são reorganizados em torno dele, separando os menores à esquerda e os maiores à direita.
Durante a execução, observe que:

- O algoritmo escolhe um elemento do vetor como pivô
- Os elementos menores que o pivô são posicionados à esquerda
- Os elementos maiores que o pivô são posicionados à direita
- Após o particionamento, o pivô fica na posição correta
- O mesmo processo é repetido para os subvetores à esquerda e à direita
- O processo continua até que todo o vetor esteja completamente ordenado

Essa análise detalhada permite entender claramente a lógica do algoritmo e como ele evolui até que o vetor esteja completamente ordenado.

Considere o vetor: [5, 3, 4, 1, 2, 8, 7, 6]

Passagem 1 (pivô = 5)

Comparações e trocas:
(5,3) → menor → [3,...]
(5,4) → menor → [3,4,...]
(5,1) → menor → [3,4,1,...]
(5,2) → menor → [3,4,1,2,...]
(5,8) → maior → ok
(5,7) → maior → ok
(5,6) → maior → ok

Posiciona pivô:
→ [3,4,1,2,5,8,7,6]

Pivô (5) já está na posição correta

Subvetor esquerdo [3,4,1,2]

Passagem 2 (pivô = 3)

Comparações:
(3,4) → maior → ok
(3,1) → menor → [1,...]
(3,2) → menor → [1,2,...]

Posiciona pivô:
→ [1,2,3,4,5,8,7,6]

Pivô (3) já está correto

Subvetor esquerdo [1,2]

Passagem 3 (pivô = 1)

Comparações:
(1,2) → maior → ok

Posiciona pivô:
→ [1,2,3,4,5,8,7,6]

Subvetor ordenado

Subvetor direito [8,7,6]

Passagem 4 (pivô = 8)

Comparações:
(8,7) → menor → [7,...]
(8,6) → menor → [7,6,...]

Posiciona pivô:
→ [1,2,3,4,5,7,6,8]

Pivô (8) já está correto

Subvetor [7,6]

Passagem 5 (pivô = 7)

Comparações:
(7,6) → menor → [6,...]

Posiciona pivô:
→ [1,2,3,4,5,6,7,8]

Vetor ordenado

Resultado final: [1, 2, 3, 4, 5, 6, 7, 8]

Implementação em C ANSI

```
#include <stdio.h>

#define TAM 8

// Função para trocar valores
void troca(int *a, int *b){
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}

// Função de particionamento
int particiona(int *v, int inicio, int fim){

    int pivo, i, j;

    pivo = v[fim];
```

```
    i = inicio - 1;

    for(j = inicio; j < fim; j++){
        if(v[j] < pivo){
            i++;
            troca(&v[i], &v[j]);
        }
    }

    troca(&v[i + 1], &v[fim]);

    return i + 1;
}

// Função recursiva do Quick Sort
void quickSort(int *v, int inicio, int fim){

    int posPivo;

    if(inicio < fim){
        posPivo = particiona(v, inicio, fim);

        quickSort(v, inicio, posPivo - 1);
        quickSort(v, posPivo + 1, fim);
    }
}

int main(void){

    int v[TAM];
    int i;

    // Entrada de dados
    for(i = 0; i < TAM; i++){
        printf("Digite o %i numero: ", i+1);
        scanf("%i", &v[i]);
    }

    // Quick Sort
    quickSort(v, 0, TAM - 1);

    // Saída de dados
    printf("\nVetor ordenado:\n");
    for(i = 0; i < TAM; i++){
        printf("%i ", v[i]);
    }

    return 0;
}
```